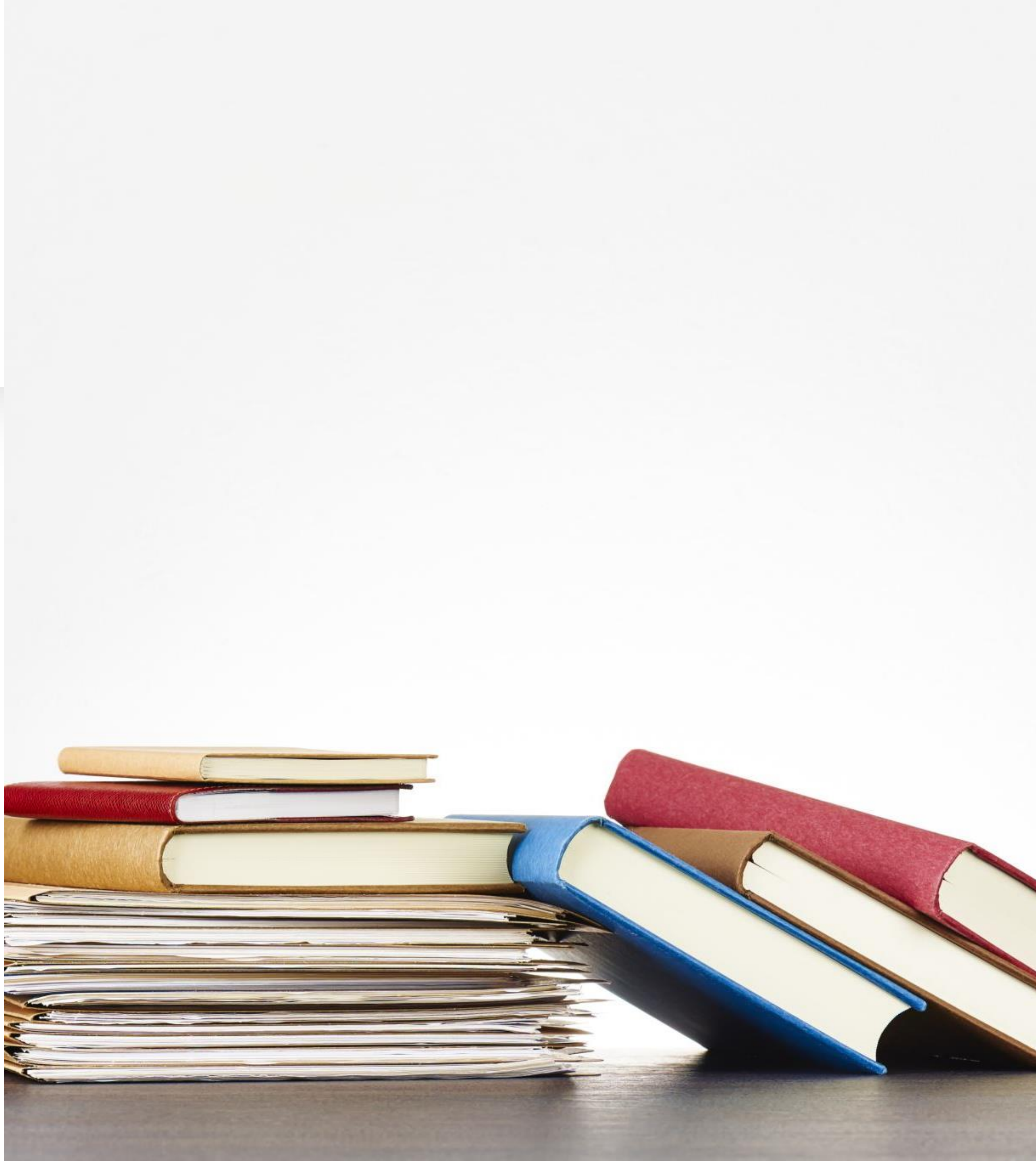
# Readers Haven Digitalization Project

Week1

LZPMPC005L Programming Clinic.

# The Problem: Modernizing "Readers Haven"

- **The Business Case:** "Readers Haven" is a legacy bookstore operating for 30 years. Current sales are declining due to the lack of an efficient digital system.
- **The Current Limitation:** The owner, Mrs. Smith, relies on a manual, handwritten ledger. This is inefficient, prone to human error, and lacks search/filter capabilities.
- **Key Objectives:**
- **Digitization:** Convert 30 years of manual data into a structured, digital database.
- **Operational Efficiency:** Implement backend logic to allow quick inventory retrieval and filtering by author name.
- **Scalability:** Build a foundation that allows "Readers Haven" to compete with modern online platforms.



# Technical Approach & Architecture

**Core Framework:** Django (Python)

*Why:* Robust Object-Relational Mapping (ORM) and built-in security features.

**Data Strategy:** API-First Design

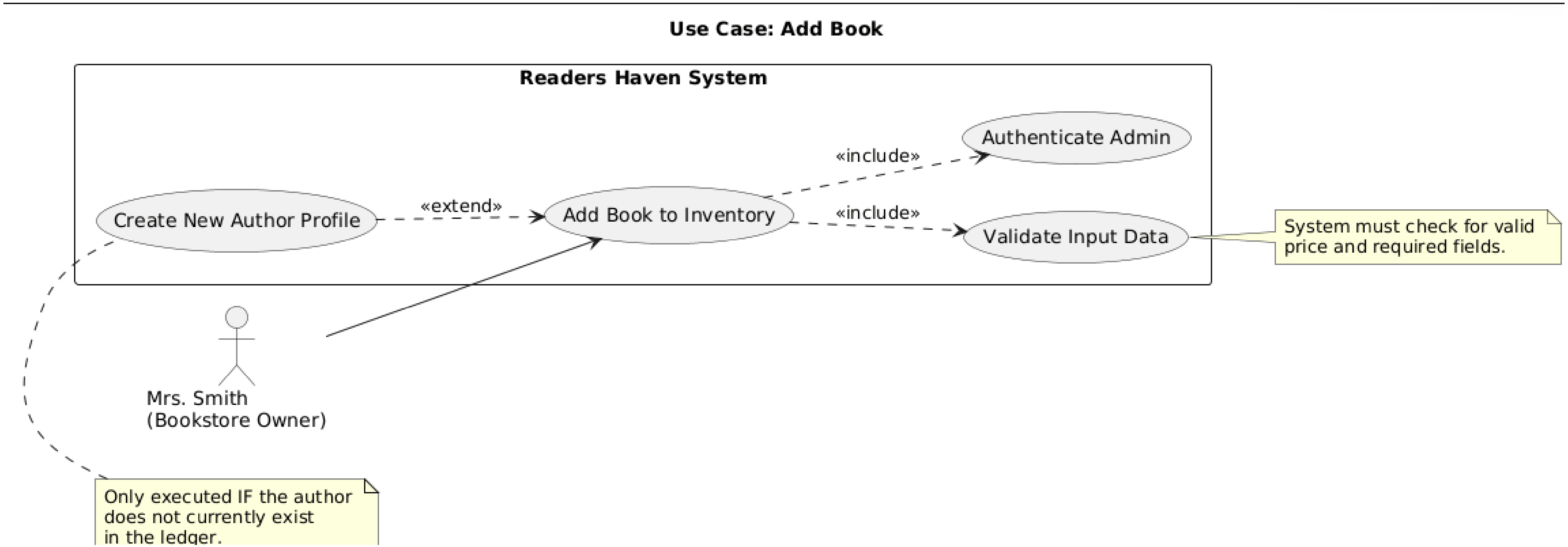
*Why:* Ensures modularity; the backend provides data via JSON, independent of any specific frontend UI.

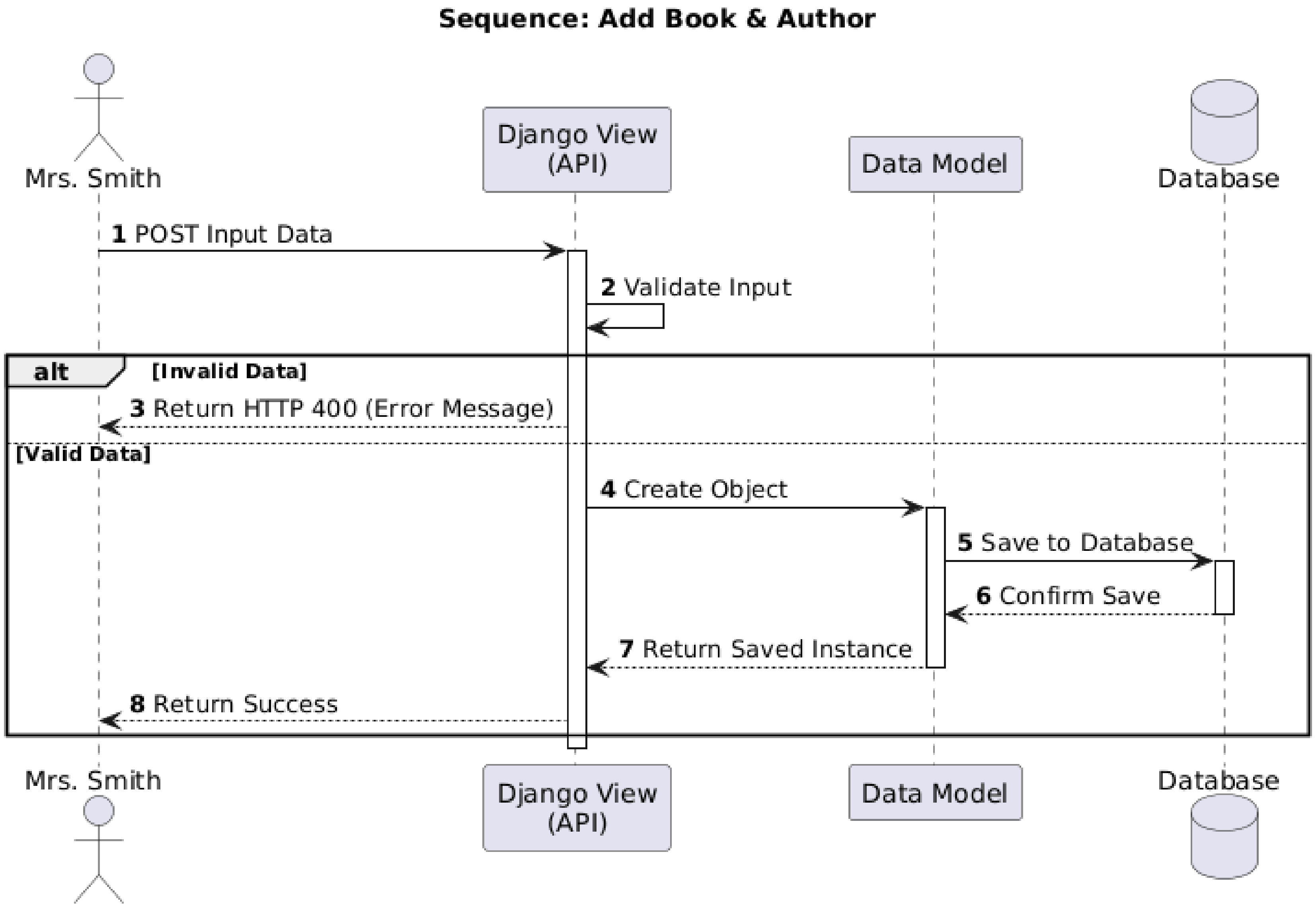
**Programming Philosophy:** Object-Oriented Programming (OOP)

*Why:* Used classes to model Book and Author as real-world objects, making the inventory system maintainable, reusable, and easy to extend.

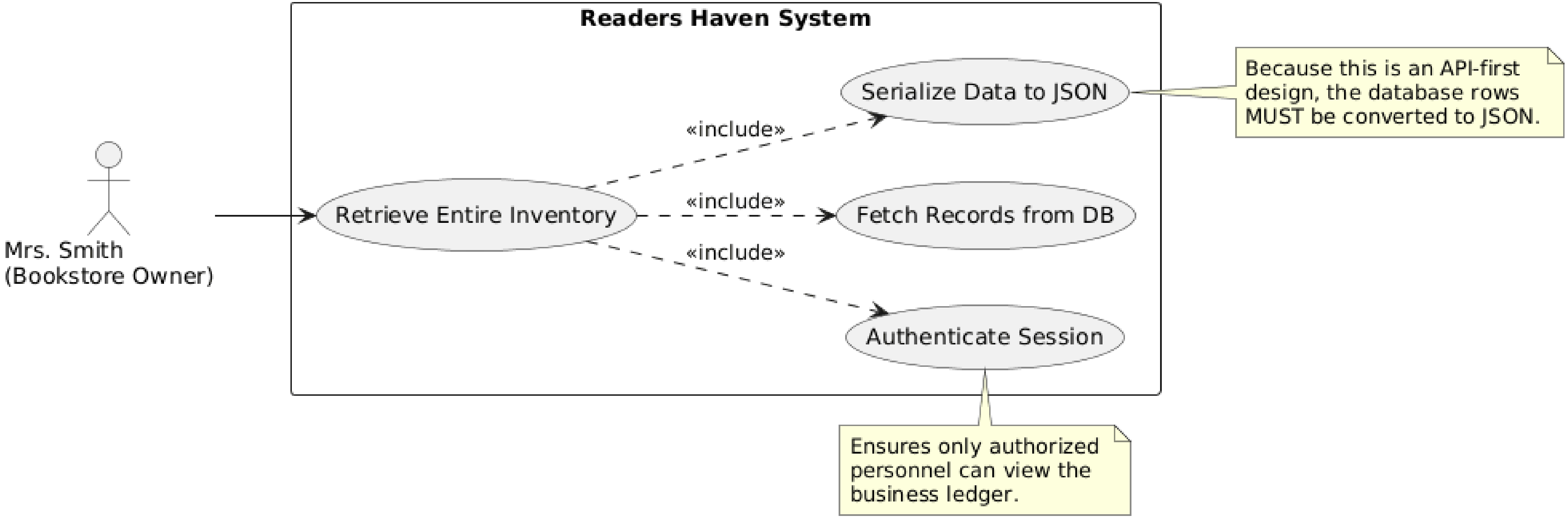
**Communication:** RESTful JSON responses for all CRUD operations (Create, Read, Update, Delete).

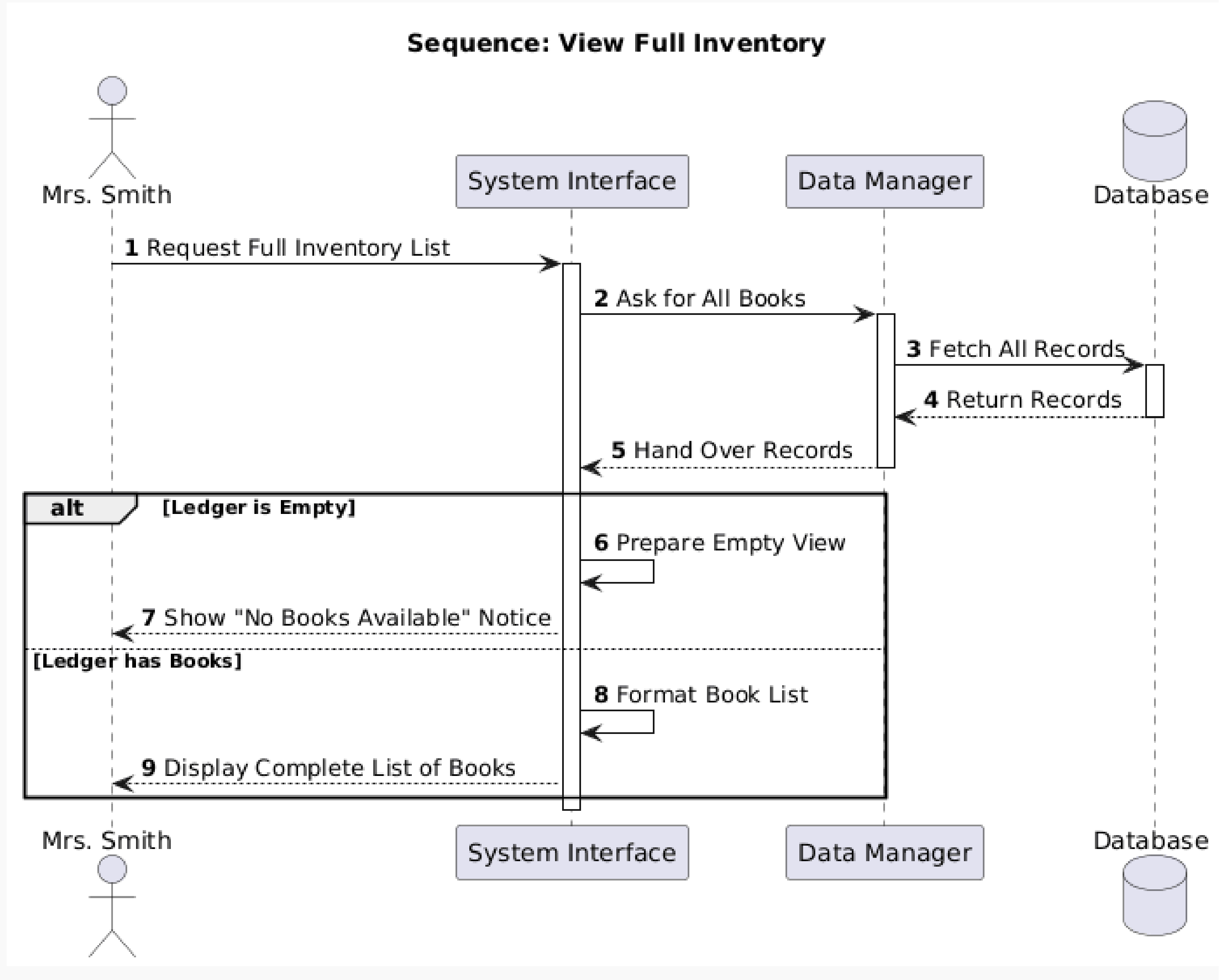
# UML DIAGRAMS for Add books and authors to the inventory

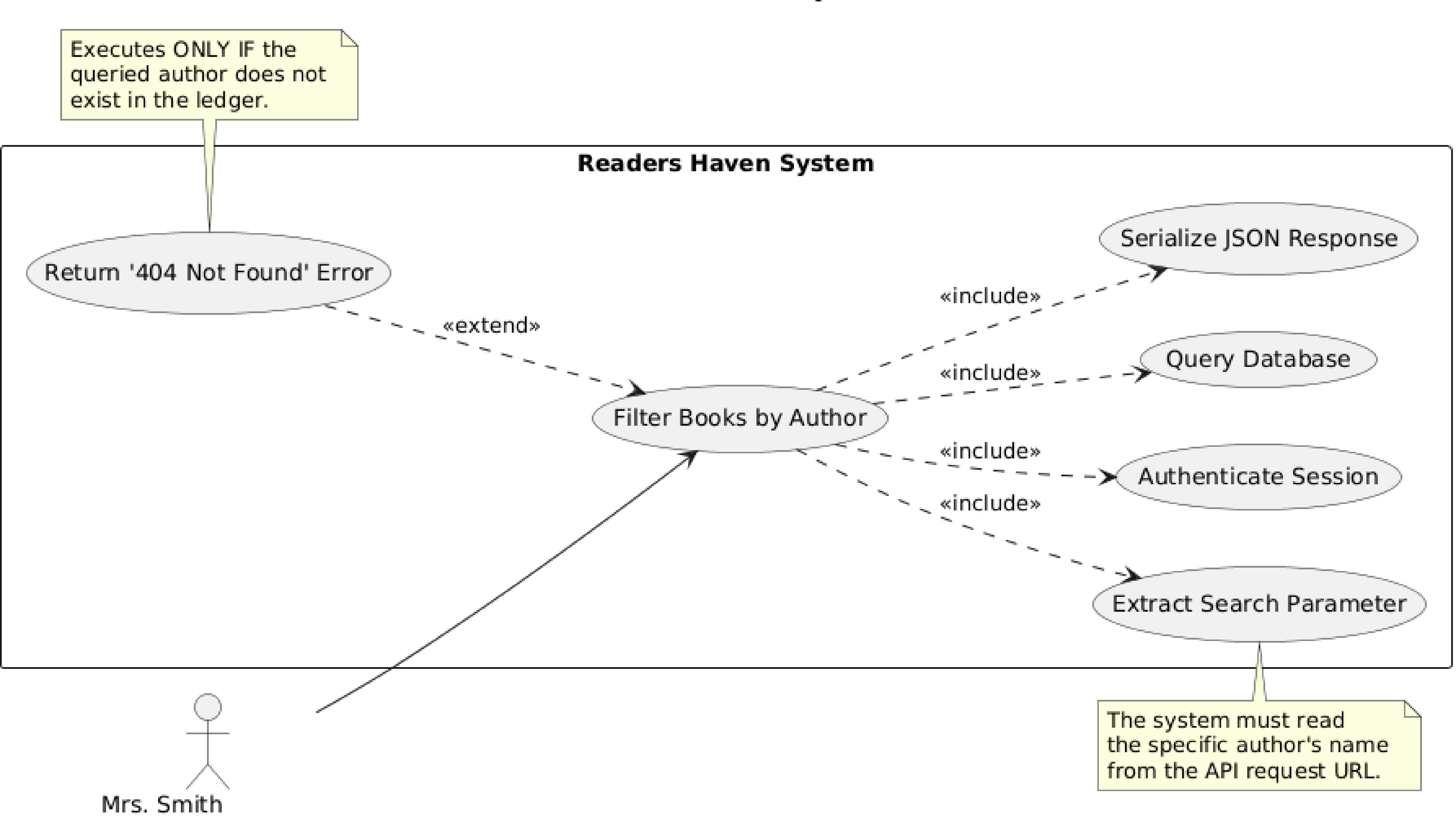


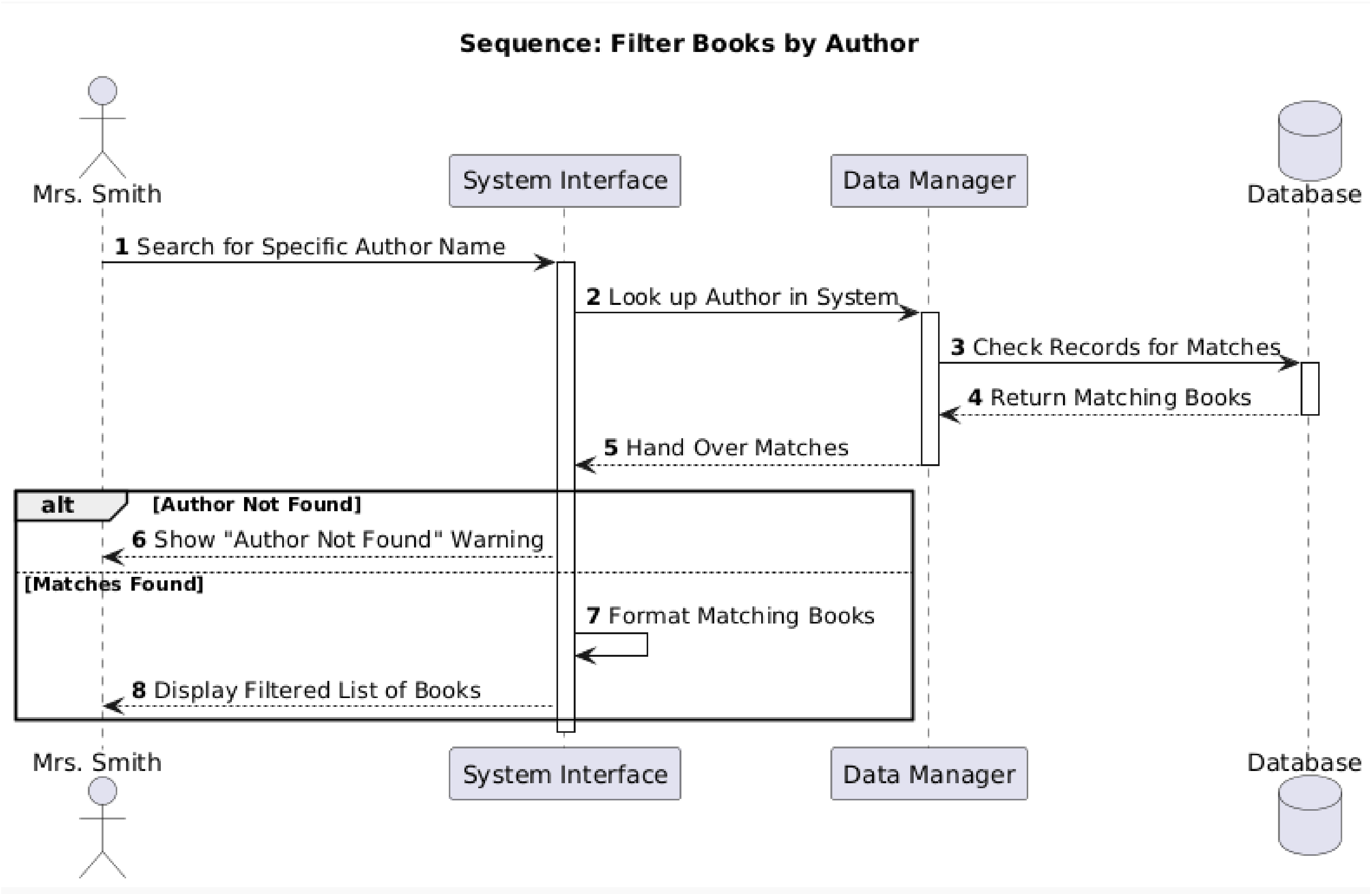

Sequence: Add Book & Author
Mrs. Smith
Django View (API)
Data Model
Database
1 POST Input Data
2 Validate Input
alt
[Invalid Data]
3 Return HTTP 400 (Error Message)
[Valid Data]
4 Create Object
5 Save to Database
6 Confirm Save
7 Return Saved Instance
8 Return Success
Mrs. Smith
Django View (API)
Data Model
Database

Use Case: Retrieve Entire Inventory




Sequence: View Full Inventory
Mrs. Smith
System Interface
Data Manager
Database
1 Request Full Inventory List
2 Ask for All Books
3 Fetch All Records
4 Return Records
5 Hand Over Records
alt
[Ledger is Empty]
6 Prepare Empty View
7 Show "No Books Available" Notice
[Ledger has Books]
8 Format Book List
9 Display Complete List of Books


Executes ONLY IF the queried author does not exist in the ledger.
Readers Haven System
Return '404 Not Found' Error
«extend»
Filter Books by Author
Serialize JSON Response
«include»
Query Database
«include»
Authenticate Session
«include»
Extract Search Parameter
«include»
Mrs. Smith
The system must read the specific author's name from the API request URL.


Sequence: Filter Books by Author
Mrs. Smith
System Interface
Data Manager
Database
1 Search for Specific Author Name
2 Look up Author in System
3 Check Records for Matches
4 Return Matching Books
5 Hand Over Matches
alt
[Author Not Found]
6 Show "Author Not Found" Warning
[Matches Found]
7 Format Matching Books
8 Display Filtered List of Books
Mrs. Smith
System Interface
Data Manager
Database

# Challenges Faced

- Data Entry Complexity: Early in the project, type in and remember author ID numbers. This was frustrating and often created errors when entering data.

- Inflexible Search System: A regular search box can only look for what it has been told to find (like Title or Price). If the bookstore starts tracking something new later on, for instance "genre", the search will fail and you will have to have a programmer to fix it.

# Solutions that were created

- Smart Typing Assistant: I created a new version of the solution that only requires the librarian to enter the author's full name, and the solution operates in a background mode similar to a smart assistant - it will find and connect the author to the existing books, or create a new entry for the author if no records are found for the author.

- A Future-Proofed Search Engine: I have built a search engine that reads the way the bookstore organizes its information rather than using "hard coded" rules; therefore, when Mrs. Smith adds another category to her inventory, the search engine recognizes the new category on its own without requiring me to write any new code.